LAB 09 On-device persistence: preferences and relational SQLite with Drift

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Persist small key-value data and build a relational table with reactive CRUD that survives app restarts.

Core concepts. SharedPreferencesAsync, SQL tables and primary keys, Drift typed schema, code generation, insert/select/delete, reactive streams and migrations.

Before you start. Use only non-sensitive preferences for small demo data. For real confidential data use appropriate secure-storage design and encryption.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab09
cd mad_lab09
flutter pub add shared_preferences drift drift_flutter
flutter pub add --dev drift_dev build_runner
```

Solved task 1 | Persist a reading goal with SharedPreferencesAsync

Problem and objective. Increment the number of completed reading sessions and retain the value after fully closing and reopening the app. This exercise uses the current asynchronous preferences API.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: ReadingGoalPage()));

class ReadingGoalPage extends StatefulWidget {
  const ReadingGoalPage({super.key});
  @override
  State<ReadingGoalPage> createState() => _ReadingGoalPageState();
}

class _ReadingGoalPageState extends State<ReadingGoalPage> {
  final prefs = SharedPreferencesAsync();
  int sessions = 0;
  bool loading = true;

  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    final value = await prefs.getInt('reading_sessions') ?? 0;
    if (!mounted) return;
    setState(() { sessions = value; loading = false; });
  }

  Future<void> addSession() async {
    final next = sessions + 1;
    await prefs.setInt('reading_sessions', next);
    if (mounted) setState(() => sessions = next);
  }

  Future<void> reset() async {
    await prefs.remove('reading_sessions');
    if (mounted) setState(() => sessions = 0);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Reading tracker')),
    body: Center(child: loading
      ? const CircularProgressIndicator()
      : Column(mainAxisSize: MainAxisSize.min, children: [
          Text('$sessions sessions completed',
            style: Theme.of(context).textTheme.headlineSmall),
          FilledButton(onPressed: addSession,
            child: const Text('Complete session')),
          TextButton(onPressed: reset, child: const Text('Reset')),
        ])),
  );
```

```
}
```

How the solution works. SharedPreferencesAsync reads and writes a platform key-value store without a stale in-process cache. initState starts loading the saved value, and mounted checks protect against an asynchronous reply arriving after the widget is disposed.

Expected output. Increment the counter, force-close the app and reopen; the saved number returns. Reset removes the stored key.

Check your work. Confirm values survive a process restart but not necessarily an app uninstall or device data clearance.

Solved task 2 | Drift database: add, watch and delete notes

Problem and objective. Build a local relational SQLite-backed notes table, demonstrate reactive streams and generate type-safe database bindings. The generated file database.g.dart is produced by build_runner, not typed by students.

lib/database.dart

```
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';

part 'database.g.dart';

class Notes extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get content => text().withLength(min: 1, max: 200)();
}

@DriftDatabase(tables: [Notes])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(driftDatabase(name: 'csc303_notes'));
  @override
  int get schemaVersion => 1;

  Stream<List<Note>> watchNotes() => select(notes).watch();
  Future<int> createNote(String content) =>
      into(notes).insert(NotesCompanion.insert(content: content));
  Future<int> deleteNote(int id) =>
      (delete(notes)..where((t) => t.id.equals(id))).go();
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'database.dart';

void main() => runApp(const MaterialApp(home: NotesPage()));

class NotesPage extends StatefulWidget {
  const NotesPage({super.key});
  @override
  State<NotesPage> createState() => _NotesPageState();
}

class _NotesPageState extends State<NotesPage> {
  final db = AppDatabase();
  final input = TextEditingController();

  @override
  void dispose() { input.dispose(); db.close(); super.dispose(); }

  Future<void> addNote() async {
    final text = input.text.trim();
    if (text.isEmpty || text.length > 200) return;
    await db.createNote(text);
    input.clear();
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Drift notes')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(12), child: Row(children: [
        Expanded(child: TextField(controller: input,
          decoration: const InputDecoration(labelText: 'New note'))),
        IconButton(onPressed: addNote, icon: const Icon(Icons.add)),
      ])),
      Expanded(child: StreamBuilder<List<Note>>(
```

```
        stream: db.watchNotes(),
        builder: (context, snapshot) {
          if (snapshot.hasError) return Text('Error: ${snapshot.error}');
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final values = snapshot.data!;
          return ListView.builder(
            itemCount: values.length,
            itemBuilder: (_, i) => ListTile(
              title: Text(values[i].content),
              trailing: IconButton(
                icon: const Icon(Icons.delete_outline),
                onPressed: () => db.deleteNote(values[i].id)),
              ),
            );
        },
      )),
    ]),
  );
}
```

TERMINAL — generate before running

```
dart run build_runner build --delete-conflicting-outputs
flutter analyze
flutter run
```

How the solution works. The Notes class describes an SQL table with a primary key and constrained text. Drift generates _$AppDatabase and data class Note, enforcing types at compile time. The watchNotes query emits whenever underlying table data changes; StreamBuilder rebuilds the visible rows. schemaVersion must be incremented and migrations implemented for later schema changes.

Expected output. The app can add and delete note records; notes remain after force-closing and restarting the app.

Check your work. Confirm the generated database.g.dart exists. Task 2 covers create, watch/read and delete. Complete the update method in the following extension page to demonstrate all four CRUD operations.

FIELD NOTE • Drift generated code is required to build, and must be regenerated after a schema change. Generated Dart code may be committed to a team repository if your agreed policy requires reproducible builds; database files and credentials should not be committed.

Scenario-based practice — unsolved

Scenario 1. Offline expense database. Design tables for Expense and Category with a category foreign key and report monthly totals. Identify required schema constraints and migration risks.

Scenario 2. Preference migration. A preference key changes from themeDark (bool) to themeMode (string). Describe how to migrate existing user data safely and handle missing/invalid old values.

Technical viva — justify your reasoning

1. Why not store sensitive passwords in SharedPreferences?
2. What does a primary key guarantee?
3. How do stream-based queries differ from polling a database?
4. Why can changing Drift schema without migration destroy or hide existing data?
5. When is a key-value store preferable to SQL?
6. What is build_runner generating and why does the app need it?

Submission and self-check

Submit persistence test evidence (before/after restart), generated-code command, the database schema and a commit with a working notes UI.

CSC303 / NOVEMBER 2026 REVISION

LAB 09 EXTENSION | Complete the Drift CRUD requirement

Task 2 currently implements create, read/watch and delete. Treat its description as **CRD** until the update method below is added. The original stream and table do not need to be rewritten.

Step 1 | Add an update method to lib/database.dart

```
Future<int> updateNote(int id, String content) =>
    (update(notes)..where((t) => t.id.equals(id))).write(
      NotesCompanion(content: Value(content)),
    );
```

Place this method inside `AppDatabase`, alongside `createNote` and `deleteNote`. The existing `package:drift/drift.dart` import supplies `Value`.

Step 2 | Add an edit dialog inside _NotesPageState

```
Future<void> editNote(Note note) async {
  var draft = note.content;
  final result = await showDialog<String>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Edit note'),
      content: TextFormField(
        initialValue: note.content,
        onChanged: (value) => draft = value,
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Cancel')),
        FilledButton(onPressed: () => Navigator.pop(dialogContext, draft),
            child: const Text('Save')),
      ],
    ),
  );
  if (!mounted || result == null) return;
  final changed = result.trim();
  if (changed.isEmpty || changed.length > 200) return;
  await db.updateNote(note.id, changed);
}
```

In the existing ListTile builder, add `onTap: () => editNote(values[i]),`. This extends, rather than replaces, the preceding Lab 09 source code.

Checks that demonstrate all four operations

- Create two notes; verify both appear in the stream.
- Tap a note, edit and save; verify its visible text changes without `setState` for the list.
- Cancel editing and verify no update; test empty/201-character input.
- Delete one note, force-close the app, reopen and confirm persisted state.

SQLite/Drift is the required local relational storage path in this task. Preferences remain appropriate only for small, non-confidential values.